TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TỔNG HỢP TOÀN BỘ ĐỒ ÁN OOP (ĐIỂM 10/10)

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (OOP JAVA) • BẢN TỔNG HỢP TOÀN DIỆN

Toàn bộ 4 Thành viên nhóm | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

TÀI LIỆU CHI TIẾT DÀNH CHO THÀNH VIÊN HỌC & BẢO VỆ VẤN ĐÁP (ĐIỂM 10/10)

> **Phân công phụ trách:** Thành viên 1

> **Mục tiêu:** Hiểu sâu từng dòng code, phân biệt rõ code chính - code phụ trợ, nắm chắc kỹ thuật chính - kỹ thuật phụ và trả lời trôi chảy mọi câu hỏi vấn đáp của Thầy Hiếu.

1. DANH MỤC CÁC TẬP TIN (FILES) QUẢN LÝ TRONG PHẦN 1

Phần 1 gồm tổng cộng **9 tập tin mã nguồn (.java)**:

1.1. Các tập tin mã nguồn chính (Core Files) - 7 Lớp

1. `SanPhamThoiTrang.java` : Lớp cha trừu tượng (Abstract Base Class), đóng vai trò xương sống cho toàn bộ mô hình hàng hóa.
2. `AoKhoac.java` : Lớp con chuyên biệt quản lý áo khoác (có/không có nón, bảo hành 12 tháng).
3. `AoThun.java` : Lớp con chuyên biệt quản lý áo thun (kích cỡ S, M, L, XL).
4. `QuanJean.java` : Lớp con chuyên biệt quản lý quần jean (kiểu dáng Slim, Baggy, Ống rộng).
5. `PhuKien.java` : Lớp con chuyên biệt quản lý phụ kiện (chất liệu da, kim loại, bảo hành 6 tháng).
6. `VayDam.java` : Lớp con chuyên biệt quản lý váy đầm thời trang nữ (chất liệu vải voan, lụa, ren).
7. `GiayDep.java` : Lớp con chuyên biệt quản lý giày dép (size số từ 35 đến 45, bảo hành 3 tháng).

1.2. Các tập tin phụ trợ & Giao tiếp (Supporting & Interfaces) - 2 Interface

8. `IBaoHanh.java` : Interface định nghĩa năng lực bảo hành chính hãng (`printWarrantyCard` , `getWarrantyPeriod`).
9. `IGiamGia.java` : Interface định nghĩa khả năng chiết khấu giá sản phẩm (`calculateAfterDiscount`).

2. BẢNG PHÂN LOẠI KĨ THUẬT LẬP TRÌNH: CHÍNH VS PHỤ TRỢ

Phân loại	Tên kĩ thuật	Vị trí xuất hiện trong mã nguồn	Vai trò & Ý nghĩa kỹ thuật
Kỹ thuật chính	**Tính Trừu tượng (Abstraction)**	`abstract class SanPhamThoiTrang` , hàm `getThongTinRieng()` , `getStringSaveFile()`	Tạo khuôn mẫu chuẩn, ngăn khởi tạo đối tượng chung chung bằng lệnh `new` , bắt buộc lớp con phải cụ thể hóa.
Kỹ thuật chính	**Tính Kế thừa (Inheritance)**	`extends SanPhamThoiTrang` , `super(Type, MaSP, TenSP, GiaSP)`	Tái sử dụng 100% thuộc tính chung của sản phẩm, hình thành cây phân cấp danh mục hàng hóa.

Kỹ thuật chính	**Tính Đa hình (Polymorphism)**	`@Override` các phương thức trừu tượng và Interface `IBaoHanh`	Tại runtime, cùng lời gọi hàm nhưng mỗi lớp con tự thực hiện hành vi riêng biệt (Áo khoác in 12 tháng, Giày in 3 tháng).
Kỹ thuật chính	**Tính Đóng gói (Encapsulation)**	`private` fields, các getter/setter có validation	Che giấu trạng thái nội tại, kiểm soát giá bán không âm, chuẩn hóa dữ liệu hợp lệ.
Kỹ thuật chính	**Hiện thực Interface**	`implements IBaoHanh`, `implements IGiamGia`	Biểu diễn quan hệ "CAN-DO" (năng lực bảo hành). Tách rời hành vi bảo hành ra khỏi bản chất sản phẩm.
Kỹ thuật phụ trợ	**Thành phần Static (Tĩnh)**	`public static final TI_LE_VAT`, `tongSoLuongSanPhamDaTao`	Hằng số dùng chung toàn hệ thống (10% VAT) và biến đếm tổng đối tượng cấp lớp chia sẻ 1 vùng nhớ duy nhất.
Kỹ thuật phụ trợ	**Tiện ích hiển thị Console**	`formatTien()`, `formatCell()`, `inTieuDeBang()`, `inChanBang()`	Canh lề, đóng khung viền bảng như Excel, cắt chuỗi tự động (`...`) chống vỡ bố cục khi tên quá dài.
Kỹ thuật phụ trợ	**Làm sạch dữ liệu (Sanitization)**	`.replace(",", " ")`, `.toUpperCase()`	Khử dấu phẩy chống vỡ định dạng CSV khi lưu file text, chuẩn hóa size áo hoa thường.

3. PHÂN TÍCH CHI TIẾT TỪNG ĐOẠN CODE: CHÍNH VS PHỤ TRỢ

3.1. Tại tập tin `SanPhamThoiTrang.java`

Đoạn Code A: Khai báo lớp trừu tượng và thuộc tính Static

```
public abstract class SanPhamThoiTrang implements IGiamGia {
    public static final double TI_LE_VAT = 0.10;
    private static int tongSoLuongSanPhamDaTao = 0;

    private String Type;
    private String MaSP;
    private String TenSP;
    private double GiaSP;
```

- **Kỹ thuật chính:**
- `public abstract class` : Khai báo lớp trừu tượng. Lớp này không thể khởi tạo thực thể trực tiếp (`new SanPhamThoiTrang()` sẽ báo lỗi biên dịch).
- `private ...` : Đóng gói an toàn toàn bộ thuộc tính, ngăn chặn việc gán sai từ lớp khác.
- **Kỹ thuật phụ trợ:**
- `public static final double TI_LE_VAT = 0.10` : Hằng số tĩnh đại diện 10% thuế VAT. Không bị nhân bản bộ nhớ theo từng đối tượng.

- `private static int tongSoLuongSanPhamDaTao = 0` : Biến tĩnh cấp lớp để theo dõi số lượng sản phẩm được tạo trong vòng đời ứng dụng.

Đoạn Code B: Hàm khởi tạo (Constructor) có tham số

```
public SanPhamThoiTrang(String Type, String MaSP, String TenSP, double GiaSP) {
        this.Type = (Type != null) ? Type : "ChuaPhanLoai";
        this.MaSP = (MaSP != null) ? MaSP : "";
        this.TenSP = (TenSP != null) ? TenSP : "";
        this.GiaSP = (GiaSP >= 0) ? GiaSP : 0;
        tongSoLuongSanPhamDaTao++;
    }
```

- **Kỹ thuật chính:** Constructor có tham số giúp thiết lập trạng thái khởi tạo cho đối tượng ngay khi cấp phát vùng nhớ.
- **Kỹ thuật phụ trợ (Phòng ngừa lỗi runtime):**
- Toán tử 3 ngôi `(GiaSP >= 0) ? GiaSP : 0` : Ràng buộc đơn giá không được âm.
- `(Type != null) ...` : Khử giá trị `null` thành chuỗi rỗng để chống ngoại lệ `NullPointerException` .
- `tongSoLuongSanPhamDaTao++` : Tự động tăng biến đếm tĩnh mỗi khi có đối tượng mới được tạo.

Đoạn Code C: Phương thức trừu tượng (Abstract Methods)

```
public abstract String getStringSaveFile();
    public abstract String getThongTinRieng();
```

- **Kỹ thuật chính:** Khai báo phương thức trừu tượng không có thân hàm `{}` . Đây là "hợp đồng bắt buộc": Mọi lớp con kế thừa (`AoKhoac` , `AoThun` ...) **bắt buộc phải override** 2 hàm này, nếu không chương trình sẽ không thể biên dịch.

Đoạn Code D: Kỹ thuật phụ trợ định dạng hiển thị Console

```
public static String formatTien(double amount) {
        DecimalFormatSymbols symbols = new DecimalFormatSymbols();
        symbols.setGroupingSeparator('.');
        DecimalFormat df = new DecimalFormat("#,##0", symbols);
        return df.format(Math.round(amount)) + " VND";
    }

    public static String formatCell(String text, int maxLen) {
        if (text == null) return "";
        if (maxLen <= 3) return (text.length() > maxLen) ? text.substring(0, maxLen) : text;
        if (text.length() > maxLen) {
            return text.substring(0, maxLen - 3) + "...";
        }
        return text;
    }
```

- **Kỹ thuật phụ trợ:**
- `formatTien` : Dùng `DecimalFormat` của Java để chuyển đổi `1800000` thành `1.800.000 VND` (chuẩn hiển thị tiền tệ Việt Nam).
- `formatCell` : Hàm cắt chuỗi thông minh. Nếu tên sản phẩm dài hơn độ rộng cột (ví dụ 22 ký tự), hàm tự động cắt và thêm dấu `...` , giúp bảng danh sách trên Console luôn thẳng hàng tuyệt đối.

3.2. Tại tập tin lớp con `AoKhoac.java`

```
public class AoKhoac extends SanPhamThoiTrang implements IBaoHanh {
    private boolean hasHat;

    public AoKhoac(String Type, String MaSP, String TenSP, double GiaSP, boolean hasHat) {
        super(Type, MaSP, TenSP, GiaSP);
        this.hasHat = hasHat;
    }

    @Override
    public String getThongTinRieng() {
        return "Co non: " + (this.hasHat ? "Co" : "Khong");
    }

    @Override
    public void printWarrantyCard() {
        System.out.println("+------------------------------------------------------------+");
        System.out.println("|                    PHIEU BAO HANH                          |");
        System.out.println("+------------------------------------------------------------+");
        System.out.printf("| Ten san pham      : %-33s |\n", getTenSP());
        System.out.printf("| Ma san pham       : %-33s |\n", getMaSP());
        System.out.printf("| Thoi han bao hanh : %-33s |\n", getWarrantyPeriod() + " thang");
        System.out.println("+------------------------------------------------------------+");
    }

    @Override
    public int getWarrantyPeriod() {
        return 12;
    }

    @Override
    public String getStringSaveFile() {
        return getType() + "," + getMaSP() + "," + getTenSP() + "," + getGiaBan() + "," + this.hasHat;
    }
}
```

- **Kỹ thuật chính (Kế thừa & Constructor Chaining):**
- `extends SanPhamThoiTrang` : Thiết lập quan hệ kế thừa.
- `super(Type, MaSP, TenSP, GiaSP)` : Lời gọi constructor lớp cha. Dòng này **bắt buộc phải đứng đầu tiên** trong constructor lớp con để đảm bảo phần thuộc tính kế thừa được khởi tạo trước.
- `this.hasHat = hasHat` : Khởi tạo thuộc tính riêng của áo khoác.
- **Kỹ thuật chính (Đa hình qua Ghi đè phương thức):**
- `@Override getThongTinRieng()` : Hiện thực phương thức trừu tượng từ lớp cha.
- `@Override getWarrantyPeriod()` : Trả về thời hạn bảo hành 12 tháng từ interface `IBaoHanh` .
- `@Override printWarrantyCard()` : Tự đóng khung in phiếu bảo hành riêng cho Áo khoác.
- **Kỹ thuật phụ trợ:**
- `@Override getStringSaveFile()` : Xuất định dạng 5 cột nối bằng dấu phẩy để phục vụ việc ghi vào file `products.txt` .

3.3. So sánh logic các lớp con chuyên biệt còn lại

10. **`AoThun.java`**:

- *Code chính:* `private String size;`

- *Code phụ trợ:* Trong setter `setSize(String size)`, kiểm tra nếu chuỗi nhập vào là `s, m, l, xl` thì dùng `.toUpperCase()` đổi thành chữ hoa. Nếu nhập sai, ép về kích thước mặc định `"M"`.

11. **`QuanJean.java`**:

- *Code chính:* `private String Style;` (Kiểu dáng: Skinny, Baggy, Ống rộng).
- *Code phụ trợ:* Tự động gọi `.replace(",", " ")` để làm sạch chuỗi kiểu dáng khi người dùng nhập có dấu phẩy.

12. **`PhuKien.java`**:

- *Code chính:* `implements IBaoHanh`, `getWarrantyPeriod() { return 6; }` (Bảo hành 6 tháng).

13. **`VayDam.java`**:

- *Code chính:* `private String chatLieuVai;` (Chất liệu vải: Voan, Lụa, Ren).

14. **`GiayDep.java`**:

- *Code chính:* `private int sizeGiay;`, `implements IBaoHanh`, `getWarrantyPeriod() { return 3; }` (Bảo hành 3 tháng).
- *Code phụ trợ:* Kiểm tra điều kiện `sizeGiay >= 35 && sizeGiay <= 45`. Nếu nằm ngoài khoảng sẽ từ chối hoặc gán size chuẩn 40.

4. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU (KÈM DÒNG CODE VÀ CÂU TRẢ LỜI ĐIỂM 10)

> **CÂU HỎI VẤN ĐÁP: Thầy chỉ vào dòng `public abstract class SanPhamThoiTrang` và hỏi: "Tại sao không để class bình thường mà phải thêm từ khóa `abstract`? Nếu thầy gọi `new SanPhamThoiTrang()` thì chuyện gì xảy ra?"**
>
> **TRẢ LỜI MẪU (10 ĐIỂM):** "Dạ thưa Thầy:
> 1. `SanPhamThoiTrang` là khái niệm trừu tượng, đại diện chung cho mọi mặt hàng thời trang. Trong thực tế bán hàng, khách hàng chỉ mua 'Áo khoác', 'Áo thun', hoặc 'Giày dép' cụ thể chứ không ai mua một 'sản phẩm thời trang' chung chung. Vì vậy lớp này phải là `abstract` để làm khuôn mẫu chuẩn cho các lớp con kế thừa.
> 2. Nếu Thầy gọi `new SanPhamThoiTrang(...)`, trình biên dịch Java sẽ **báo lỗi ngay tại thời điểm biên dịch (Compile Error)**, vì Java tuyệt đối ngăn chặn việc khởi tạo thực thể từ một lớp trừu tượng."

> **CÂU HỎI VẤN ĐÁP: Thầy chỉ vào hàm tạo của `AoKhoac` và hỏi: "Từ khóa `super(Type, MaSP, TenSP, GiaSP)` có tác dụng gì? Đổi vị trí xuống dòng thứ 2 có được không?"**
>
> **TRẢ LỜI MẪU (10 ĐIỂM):** "Dạ thưa Thầy:
> 1. Từ khóa `super(...)` dùng để gọi trực tiếp constructor 4 tham số của lớp cha `SanPhamThoiTrang`, chuyển giao các giá trị `Type`, `MaSP`, `TenSP`, `GiaSP` lên cho lớp cha khởi tạo và bảo vệ dữ liệu.
> 2. Lệnh `super(...)` **bắt buộc phải nằm ở dòng lệnh đầu tiên** của constructor lớp con. Nếu đổi xuống dòng thứ 2 hoặc phía sau, trình biên dịch Java sẽ báo lỗi cú pháp ngay lập tức."

> **CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Tại sao `AoKhoac`, `PhuKien`, `GiayDep` lại implements `IBaoHanh`, còn `AoThun`, `QuanJean` thì không?"**
>
> **TRẢ LỜI MẪU (10 ĐIỂM):** "Dạ thưa Thầy, đây là việc áp dụng đúng nguyên lý thiết kế Hướng đối tượng:
> 1. Interface thể hiện quan hệ **CAN-DO** (khả năng hành vi). Không phải mọi sản phẩm may mặc đều có chế độ bảo

hành. Quần áo thông thường như áo thun, quần jean chỉ áp dụng đổi trả lúc mua chứ không bảo hành theo tháng.
2. Chỉ những mặt hàng có giá trị hoặc phụ kiện/giày dép mới có bảo hành chính hãng. Nhóm thiết kế `IBaoHanh` độc lập để chỉ những lớp nào cần bảo hành mới implement, đảm bảo tính phân tách giao diện (Interface Segregation Principle).
3. Khi xuất hóa đơn, hệ thống chỉ cần dùng toán tử `sp instanceof IBaoHanh` để tự động in phiếu bảo hành đa hình mà không làm ảnh hưởng đến các sản phẩm khác."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào `public static final double TI_LE_VAT = 0.10;` và hỏi: "Ba từ khóa `public`, `static`, `final` ở đây có ý nghĩa gì?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:
- `public`: Cho phép mọi lớp khác trong dự án (như `ChiTietHoaDon`, `HoaDon`) có thể truy cập để tính thuế.
- `static`: Biến thuộc về mức lớp (Class-level), chỉ lưu trữ duy nhất 1 vùng nhớ chia sẻ cho toàn bộ ứng dụng, truy cập trực tiếp qua `SanPhamThoiTrang.TI_LE_VAT` mà không cần tạo đối tượng.
- `final`: Biến này là hằng số (Constant), giá trị 0.10 cố định không thể bị gán đè hay thay đổi trong suốt quá trình chạy."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào dòng `this.TenSP = TenSP.trim().replace(",", " ");` và hỏi: "Lệnh `.replace(",", " ")` để làm gì, nếu không có thì có lỗi gì không?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, đây là kỹ thuật làm sạch dữ liệu (Data Sanitization):
1. Hệ thống của nhóm lưu danh mục sản phẩm vào file text `products.txt` theo định dạng CSV (phân cách bằng dấu phẩy `,`).
2. Nếu người dùng nhập tên sản phẩm chứa dấu phẩy (ví dụ: 'Áo gió, 2 lớp, nam'), khi đọc lại file bằng lệnh `line.split(",")`, dòng đó sẽ bị tách thành nhiều hơn 5 phần tử, làm lệch các cột giá tiền và thuộc tính riêng, dẫn đến lỗi ngoại lệ đọc file.
3. Lệnh `.replace(",", " ")` chuyển dấu phẩy thành khoảng trắng, triệt tiêu 100% nguy cơ vỡ cấu trúc file text."

TÀI LIỆU CHI TIẾT DÀNH CHO THÀNH VIÊN HỌC & BẢO VỆ VẤN ĐÁP (ĐIỂM 10/10)

> **Phân công phụ trách:** Thành viên 2

> **Mục tiêu:** Nắm vững cấu trúc phân cấp đa tầng của mô hình Con người, bản chất đa hình chiết khấu qua Interface `IGiamGiaKhachHang`, phân biệt code chính và code phụ trợ, trả lời lưu loát các câu hỏi vấn đáp của Thầy Hiếu.

1. DANH MỤC CÁC TẬP TIN (FILES) QUẢN LÝ TRONG PHẦN 2

Phần 2 gồm tổng cộng **8 tập tin mã nguồn (.java)**:

1.1. Các tập tin mã nguồn chính (Core Files) - 7 Lớp

15. `Nguoi.java` : Lớp cơ sở (Base Class) đại diện cho thực thể con người trong hệ thống.
16. `KhachHang.java` : Lớp trung gian kế thừa từ `Nguoi`, quản lý thông tin khách mua hàng.
17. `KhachHangVIP.java` : Lớp con kế thừa từ `KhachHang`, hiện thực chiết khấu **20%** cho khách VIP.
18. `KhachHangThanThiet.java` : Lớp con kế thừa từ `KhachHang`, hiện thực chiết khấu **10%** và quản lý điểm tích lũy.
19. `NhanVien.java` : Lớp con kế thừa từ `Nguoi`, hiện thực chiết khấu nội bộ **50%** cho nhân sự cửa hàng.
20. `NhanVienQuanLy.java` : Lớp con kế thừa từ `NhanVien`, mở rộng quản lý phụ cấp trách nhiệm.
21. `NhaCungCap.java` : Lớp độc lập quản lý các đối tác sản xuất, cung ứng may mặc.

1.2. Tập tin phụ trợ & Giao tiếp (Supporting & Interfaces) - 1 Interface

22. `IGiamGiaKhachHang.java` : Interface định nghĩa phương thức giảm giá theo đối tượng (`applyDiscount`).

2. BẢNG PHÂN LOẠI KĨ THUẬT LẬP TRÌNH: CHÍNH VS PHỤ TRỢ

Phân loại	Tên kĩ thuật	Vị trí xuất hiện trong mã nguồn	Vai trò & Ý nghĩa kỹ thuật
Kỹ thuật chính	**Kế thừa đa tầng (Multi-level Inheritance)**	`Nguoi -> KhachHang -> KhachHangVIP`, `Nguoi -> NhanVien -> NhanVienQuanLy`	Xây dựng cây phả hệ logic: Quản lý kế thừa Nhân viên, VIP kế thừa Khách hàng. Tái sử dụng tối đa mã nguồn.
Kỹ thuật chính	**Đa hình qua Interface (Interface Polymorphism)**	`implements IGiamGiaKhachHang`, hàm `applyDiscount(double)`	Đồng nhất hành vi giảm giá cho các đối tượng khác nhau mà không cần viết hàng loạt lệnh if-else phân loại.
Kỹ thuật chính	**Tính Đóng gói (Encapsulation)**	`private String name, numberPhone`, getter/setter	Bảo vệ dữ liệu cá nhân, ngăn chặn việc gán bừa bãi số điện thoại hoặc mã thẻ từ bên ngoài.
Kỹ thuật chính	**Kiểm tra kiểu & Ép kiểu (Downcasting)**	`person instanceof IGiamGiaKhachHang` trong `HoaDon.java`	Kiểm tra động tại runtime để ép kiểu sang interface và gọi hàm chiết khấu tương ứng.
Kỹ thuật phụ trợ	**Làm sạch dữ liệu (Data Sanitization)**	`.replace(",", " ")`, `.replace(",", "")`	Loại bỏ dấu phẩy trong họ tên và SĐT để bảo toàn định

			dạng file text `users.txt` .
Kỹ thuật phụ trợ	**Kiểm tra biểu thức chính quy (Regex)**	`matches("^0[0-9]{8,10}$")`	Kiểm tra tính hợp lệ của số điện thoại Việt Nam (bắt đầu bằng số 0, dài từ 9 đến 11 số).
Kỹ thuật phụ trợ	**Định dạng bảng hiển thị Console**	`hienThiDanhSach()` , `hienThiThongTin()`	Đóng khung kẻ viền bảng, căn thẳng cột họ tên, SĐT và chức vụ trên màn hình.

3. PHÂN TÍCH CHI TIẾT TỪNG ĐOẠN CODE: CHÍNH VS PHỤ TRỢ

3.1. Tại tập tin lớp cơ sở `Nguoi.java`

```
public class Nguoi {
    private String name;
    private String numberPhone;

    public Nguoi(String name, String numberPhone) {
        this.name = (name != null) ? name.trim().replace(",", " ") : "";
        this.numberPhone = (numberPhone != null) ? numberPhone.trim().replace(",", "") : "";
    }

    public String getName() { return name; }
    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name.trim().replace(",", " ");
        }
    }

    public String getNumberPhone() { return numberPhone; }
    public void setNumberPhone(String numberPhone) {
        if (numberPhone != null) {
            this.numberPhone = numberPhone.trim().replace(",", "");
        }
    }
}
```

- **Kỹ thuật chính:** Đóng gói thuộc tính `name` và `numberPhone` ở phạm vi `private` . Mọi truy cập hay sửa đổi bắt buộc phải thông qua `getName()` và `setName()` .
- **Kỹ thuật phụ trợ:**
- `(name != null) ? ... : ""` : Chống lỗi `NullPointerException` khi khởi tạo.
- `.replace(",", " ")` : Loại bỏ dấu phẩy để dữ liệu khi ghi vào `users.txt` (dạng CSV) không bị lệch cột dữ liệu.

3.2. Tại Interface `IGiamGiaKhachHang.java`

```
public interface IGiamGiaKhachHang {
    double applyDiscount(double totalPrice);
}
```

- **Kỹ thuật chính:** Khai báo một giao tiếp trừu tượng thuần túy. Bất kỳ thực thể con người nào có quyền được giảm giá trong hệ thống đều phải hiện thực phương thức này.
- **Ý nghĩa:** Tách rời hoàn toàn nghiệp vụ chiết khấu ra khỏi cây kế thừa sinh học, cho phép cả Khách hàng và Nhân viên đều có thể áp dụng chung một cơ chế tính tiền.

3.3. Đoạn Code Khách hàng VIP `KhachHangVIP.java`

```
public class KhachHangVIP extends KhachHang implements IGiamGiaKhachHang {
    private String cardCodeVIP;

    public KhachHangVIP(String name, String numberPhone, String cardCodeVIP) {
        super(name, numberPhone, "VIP");
        this.cardCodeVIP = (cardCodeVIP != null) ? cardCodeVIP.trim().replace(",", " ") : "";
    }

    public String getCardCodeVIP() { return cardCodeVIP; }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.2);
    }
}
```

- **Kỹ thuật chính (Kế thừa & Constructor Chaining):**
- `super(name, numberPhone, "VIP")` : Gọi constructor 3 tham số của lớp cha `KhachHang` , vừa truyền họ tên và SĐT, vừa tự động gán loại khách hàng là `"VIP"` .
- `private String cardCodeVIP` : Thuộc tính mở rộng đặc trưng cho khách VIP (mã thẻ VIP).
- **Kỹ thuật chính (Ghi đè phương thức @Override):**
- `applyDiscount` : Thực hiện công thức `totalPrice - (totalPrice * 0.2)` -> Giảm trực tiếp **20%** trên tổng tiền đơn hàng.

3.4. Đoạn Code Khách hàng Thân thiết `KhachHangThanThiet.java`

```
public class KhachHangThanThiet extends KhachHang implements IGiamGiaKhachHang {
    private int diemTichLuy;

    public KhachHangThanThiet(String name, String numberPhone, int diemTichLuy) {
        super(name, numberPhone, "ThanThiet");
        this.diemTichLuy = (diemTichLuy >= 0) ? diemTichLuy : 0;
    }

    public int getDiemTichLuy() { return diemTichLuy; }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.10);
    }
}
```

- **Kỹ thuật chính:**
- `diemTichLuy` : Quản lý điểm thưởng tích lũy qua các lần mua sắm. Ràng buộc `diemTichLuy >= 0` .
- `@Override applyDiscount` : Chiết khấu ưu đãi **10%** cho khách hàng thân thiết.

3.5. Đoạn Code Phân cấp Nhân viên `NhanVien.java` & `NhanVienQuanLy.java`

```
public class NhanVien extends Nguoi implements IGiamGiaKhachHang {
    private String staffID;

    public NhanVien(String name, String numberPhone, String staffID) {
        super(name, numberPhone);
        this.staffID = (staffID != null) ? staffID.trim().replace(",", " ") : "";
    }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.5);
    }
}
```

```
public class NhanVienQuanLy extends NhanVien {
    private double phuCapTrachNhiem;

    public NhanVienQuanLy(String name, String numberPhone, String staffID, double phuCapTrachNhiem) {
        super(name, numberPhone, staffID);
        this.phuCapTrachNhiem = (phuCapTrachNhiem >= 0) ? phuCapTrachNhiem : 0;
    }

    public double getPhuCapTrachNhiem() { return phuCapTrachNhiem; }
}
```

- **Kỹ thuật chính (Kế thừa đa tầng & Tái sử dụng phương thức):**
- `NhanVien` kế thừa từ `Nguoi` và cài đặt `IGiamGiaKhachHang` với mức giảm nội bộ là **50%**.
- `NhanVienQuanLy extends NhanVien` : Quản lý kế thừa toàn bộ thuộc tính `staffID` và phương thức giảm giá 50% của `NhanVien` mà không cần viết lại dù chỉ 1 dòng code.
- Thuộc tính riêng `phuCapTrachNhiem` : Bổ sung khoản phụ cấp quản lý cho vị trí quản lý cửa hàng.

3.6. Lớp đối tác `NhaCungCap.java`

- Quản lý các đơn vị may mặc, xưởng dệt may đối tác (`maNCC` , `tenNCC` , `diaChi` , `soDienThoai`).
- Đóng gói phương thức `hienThiThongTin()` in dạng bảng cho Menu 9 trong hệ thống.

4. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU (KÈM DÒNG CODE VÀ CÂU TRẢ LỜI ĐIỂM 10)

> **CÂU HỎI VẤN ĐÁP: Thầy chỉ vào lớp `HoaDon.java` và hỏi: "Đoạn code nào thể hiện việc áp dụng đa hình chiết khấu cho khách hàng và nhân viên? Hãy giải thích cơ chế hoạt động?"**
>
> **TRẢ LỜI MẪU (10 ĐIỂM):** "Dạ thưa Thầy, đó là đoạn code trong phương thức `calculateTotalPrice()` của lớp `HoaDon`:
> ```java
> if (person instanceof IGiamGiaKhachHang) {
> total = ((IGiamGiaKhachHang) person).applyDiscount(total);
> }
> ```
> Cơ chế hoạt động như sau:
> 1. Lớp `HoaDon` chỉ giữ một con trỏ kiểu cha là `Nguoi person`.

2. Tại thời điểm chạy (Runtime), máy ảo Java kiểm tra xem đối tượng mà `person` đang trỏ tới có hiện thực interface `IGiamGiaKhachHang` hay không bằng toán tử `instanceof`.

3. Nếu có, Java sẽ kích hoạt cơ chế liên kết động (Dynamic Binding) để gọi hàm `applyDiscount` của đúng đối tượng thực tế đó: Nếu là `KhachHangVIP` sẽ giảm 20%, nếu là `KhachHangThanThiet` giảm 10%, nếu là `NhanVien` hoặc `NhanVienQuanLy` giảm 50%.

4. Nhờ đa hình, chúng em không cần viết hàng loạt lệnh if-else phức tạp để kiểm tra từng loại lớp, giúp hệ thống dễ dàng mở rộng khi có thêm loại khách hàng mới."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Tại sao `NhanVienQuanLy` lại kế thừa từ `NhanVien` mà không kế thừa trực tiếp từ `Nguoi`?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, theo đúng mô hình kế thừa hướng đối tượng:

1. Quản lý bản chất vẫn là một Nhân viên (Quan hệ **IS-A**). Quản lý cũng có mã nhân viên (`staffID`), cũng làm việc trong cửa hàng và cũng được hưởng chế độ giảm giá nội bộ 50% khi mua đồ thời trang.

2. Việc cho `NhanVienQuanLy extends NhanVien` giúp tái sử dụng lại 100% mã nguồn và các hành vi đã có của `NhanVien`, chỉ cần bổ sung thêm thuộc tính đặc thù của chức vụ là `phuCapTrachNhiem`. Nếu kế thừa trực tiếp từ `Nguoi`, chúng em sẽ bị trùng lặp mã nguồn (Code Duplication)."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào đoạn code nhập SĐT trong `Main.java`: `cPhone.matches("^0[0-9]{8,10}$")` và hỏi: "Biểu thức Regex này có ý nghĩa gì?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, đây là kỹ thuật kiểm tra tính hợp lệ của số điện thoại:

- `^0`: Bắt buộc chuỗi phải bắt đầu bằng chữ số 0 (quy chuẩn số điện thoại tại Việt Nam).
- `[0-9]{8,10}`: Theo sau số 0 phải là từ 8 đến 10 chữ số (tổng độ dài số điện thoại sẽ từ 9 đến 11 số).
- `$`: Đánh dấu kết thúc chuỗi, không cho phép chứa thêm bất kỳ ký tự chữ cái hay ký tự đặc biệt nào khác.
- Biểu thức này đảm bảo dữ liệu khách hàng lưu trong hệ thống luôn chuẩn xác."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Làm thế nào để hệ thống ngăn chặn việc tạo 2 người dùng có cùng một số điện thoại?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, trong lớp mảng `DanhSachKhachHang`, trước khi thêm một người dùng mới bằng hàm `them(Nguoi person)`, chương trình luôn gọi hàm `timKiemTheoSDT(person.getNumberPhone())`. Nếu kết quả trả về khác `null` (nghĩa là số điện thoại này đã tồn tại trong danh sách), hệ thống sẽ lập tức từ chối thêm và in thông báo lỗi, bảo toàn tính duy nhất của khóa chính định danh người dùng."

TÀI LIỆU CHI TIẾT DÀNH CHO THÀNH VIÊN HỌC & BẢO VỆ VẤN ĐÁP (ĐIỂM 10/10)

> **Phân công phụ trách:** Thành viên 3

> **Mục tiêu:** Nắm vững cấu trúc Lớp mảng danh sách đối tượng (Class Array / Collection Wrapper), cơ chế xử lý file text chuẩn hóa UTF-8 chống lỗi font tiếng Việt, cơ chế cộng dồn giỏ hàng và bản chất các thành phần Static trong Java.

1. DANH MỤC CÁC TẬP TIN (FILES) QUẢN LÝ TRONG PHẦN 3

Phần 3 gồm tổng cộng **4 tập tin mã nguồn (.java) và 3 tập tin dữ liệu (.txt)**:

1.1. Các tập tin mã nguồn chính (Core Files) - 4 Lớp

23. `DanhSachSanPham.java` : Lớp mảng chuyên biệt quản lý tập hợp toàn bộ sản phẩm trong kho hàng.
24. `DanhSachKhachHang.java` : Lớp mảng chuyên biệt quản lý danh sách người dùng (Khách hàng VIP, Thân thiết, Nhân viên, Quản lý).
25. `ChiTietHoaDon.java` : Lớp biểu diễn một dòng sản phẩm mua trong hóa đơn (gồm sản phẩm và số lượng).
26. `HoaDon.java` : Lớp quản lý đơn hàng, tính tiền đa hình, in hóa đơn và in phiếu bảo hành.

1.2. Các tập tin dữ liệu lưu trữ (Data Files) - 3 File

27. `products.txt` : File văn bản lưu trữ thông tin 6 loại sản phẩm thời trang.
28. `users.txt` : File văn bản lưu trữ thông tin khách hàng và nhân sự cửa hàng.
29. `hoadon.txt` : File văn bản lưu vết lịch sử mọi hóa đơn thanh toán và phiếu bảo hành đã xuất.

2. BẢNG PHÂN LOẠI KĨ THUẬT LẬP TRÌNH: CHÍNH VS PHỤ TRỢ

Phân loại	Tên kĩ thuật	Vị trí xuất hiện trong mã nguồn	Vai trò & Ý nghĩa kỹ thuật
Kỹ thuật chính	**Lớp mảng các đối tượng (Class Array)**	`DanhSachSanPham`, `DanhSachKhachHang`	Đóng gói mảng `ArrayList` bên trong class; không để lộ cấu trúc lưu trữ; cung cấp đầy đủ hàm CRUD (Xem, Thêm, Sửa, Xóa, Tìm kiếm).
Kỹ thuật chính	**Quan hệ Chứa đựng (Aggregation)**	`HoaDon` chứa `ArrayList<ChiTietHoaDon>`, `ChiTietHoaDon` chứa `SanPhamThoiTrang`	Mô hình hóa mối quan hệ thực tế: Một hóa đơn gồm nhiều món hàng, mỗi món gồm mặt hàng và số lượng tương ứng.
Kỹ thuật chính	**Thành phần Static (Tĩnh)**	`SanPhamThoiTrang.TI_LE_VAT`, `HoaDon.tongSoHoaDon`, `formatTien()`	Biến static lưu trữ vùng nhớ duy nhất mức lớp; phương thức static tiện ích dùng chung không cần tạo đối tượng.
Kỹ thuật chính	**Đọc/Ghi luồng File Text**	`docTuFile()`, `ghiVaoFile()`, `xuatHoaDonRaFile()`	Lưu trữ trạng thái bền vững xuống đĩa cứng, tự nạp lại dữ liệu khi khởi

			động lại chương trình.
Kỹ thuật phụ trợ	**Chuẩn hóa bảng mã UTF-8**	`StandardCharsets.UTF_8` kết hợp `InputStreamReader/OutputStreamWriter`	Chống 100% lỗi font tiếng Việt có dấu ("Áo gió ấm áp", "Váy dạ hội") trên mọi hệ điều hành Windows/Linux.
Kỹ thuật phụ trợ	**Cấu trúc Try-with-resources**	`try (BufferedReader reader = ...) { ... }`	Tự động đóng luồng (`reader.close()`) ngay khi đọc xong hoặc có lỗi ngoại lệ, chống rò rỉ bộ nhớ.
Kỹ thuật phụ trợ	**Thuật toán Tự sinh mã ID**	`taoMaTuDong(type)` trong `DanhSachSanPham.java`	Quét mã lớn nhất hiện có và tự động tăng dần (`AK001` , `AK002` ...), người dùng không cần gõ mã thủ công.
Kỹ thuật phụ trợ	**Cộng dồn số lượng giỏ hàng**	`tangSoLuong(int amount)` trong `ChiTietHoaDon.java`	Khi mua lặp lại cùng một mã hàng, hệ thống tự động tăng số lượng thay vì tạo thêm dòng mới.

3. PHÂN TÍCH CHI TIẾT TỪNG ĐOẠN CODE: CHÍNH VS PHỤ TRỢ

3.1. Lớp mảng danh sách sản phẩm `DanhSachSanPham.java`

Đoạn Code A: Đóng gói tập hợp `ArrayList`

```
public class DanhSachSanPham {
    private ArrayList<SanPhamThoiTrang> danhSach;

    public DanhSachSanPham() {
        this.danhSach = new ArrayList<>();
    }

    public ArrayList<SanPhamThoiTrang> getDanhSach() { return danhSach; }
    public int getSoLuong() { return danhSach.size(); }
    public boolean isEmpty() { return danhSach.isEmpty(); }
```

- **Kỹ thuật chính:** Tính Đóng gói (Encapsulation). Thuộc tính `danhSach` được đặt `private` . Mọi thao tác truy cập số lượng hay duyệt danh sách đều phải thông qua các phương thức của lớp mảng.

Đoạn Code B: Thuật toán Tự sinh mã ID tự động (Kỹ thuật phụ trợ nâng cao)

```
public String taoMaTuDong(String type) {
        String prefix = "SP";
        if (type != null) {
            String t = type.toLowerCase();
            if (t.equals("aokhoac")) prefix = "AK";
            else if (t.equals("quanjean") || t.equals("jeans")) prefix = "QJ";
            else if (t.equals("aothun") || t.equals("tshirt")) prefix = "AT";
```

```
            else if (t.equals("phukien") || t.equals("accessory")) prefix = "PK";
            else if (t.equals("vaydam")) prefix = "VD";
            else if (t.equals("giaydep")) prefix = "GD";
        }

        int maxId = 0;
        for (SanPhamThoiTrang p : danhSach) {
            if (p.getMaSP() != null && p.getMaSP().toUpperCase().startsWith(prefix)) {
                String numPart = p.getMaSP().substring(prefix.length());
                try {
                    int id = Integer.parseInt(numPart);
                    if (id > maxId) maxId = id;
                } catch (NumberFormatException ignored) {}
            }
        }
        return String.format("%s%03d", prefix, maxId + 1);
    }
```

- **Giải thích từng dòng:**
- `prefix` : Xác định tiền tố theo loại (`AK` , `AT` , `QJ` , `PK` , `VD` , `GD`).
- Vòng lặp `for` : Quét qua toàn bộ sản phẩm trong danh sách.
- `p.getMaSP().substring(prefix.length())` : Tách bỏ tiền tố để lấy phần số. Ví dụ `"AK007"` cắt bỏ 2 ký tự đầu lấy được chuỗi `"007"` .
- `Integer.parseInt(numPart)` : Chuyển chuỗi thành số nguyên và tìm số lớn nhất `maxId` .
- `String.format("%s%03d", prefix, maxId + 1)` : Định dạng mã mới với 3 chữ số đệm (ví dụ `maxId` là 7 thì sinh `AK008`).

Đoạn Code C: Nghiệp vụ CRUD đầy đủ

```
public boolean them(SanPhamThoiTrang sp) {
        if (sp == null || sp.getMaSP() == null || sp.getMaSP().trim().isEmpty()) return false;
        if (timKiem(sp.getMaSP()) != null) return false;
        danhSach.add(sp);
        return true;
    }

    public boolean xoa(String maSP) {
        if (maSP == null) return false;
        for (int i = 0; i < danhSach.size(); i++) {
            if (danhSach.get(i).getMaSP() != null &&
danhSach.get(i).getMaSP().equalsIgnoreCase(maSP.trim())) {
                danhSach.remove(i);
                return true;
            }
        }
        return false;
    }

    public boolean sua(String maSP, String tenMoi, double giaMoi) {
        SanPhamThoiTrang sp = timKiem(maSP);
        if (sp == null) return false;
        if (tenMoi != null && !tenMoi.trim().isEmpty()) sp.setTenSP(tenMoi.trim());
        if (giaMoi >= 0) sp.setGiaBan(giaMoi);
        return true;
    }

    public SanPhamThoiTrang timKiem(String maSP) {
        if (maSP == null) return null;
```

```
        for (SanPhamThoiTrang sp : danhSach) {
            if (sp.getMaSP() != null && sp.getMaSP().equalsIgnoreCase(maSP.trim())) return sp;
        }
        return null;
    }
```

- **Kỹ thuật chính:** Đảm bảo đầy đủ 4 thao tác chuẩn CRUD:
- **Create (Thêm):** Có kiểm tra trùng mã sản phẩm.
- **Read/Search (Tìm kiếm):** Tìm theo mã không phân biệt hoa thường.
- **Update (Sửa):** Tìm và cập nhật tên, đơn giá.
- **Delete (Xóa):** Tìm vị trí phần tử và gọi `remove(i)` .

3.2. Kỹ thuật Đọc/Ghi File Text chuẩn UTF-8

```
public void docTuFile(String fileName) {
        File file = new File(fileName);
        if (!file.exists()) return;

        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.trim().isEmpty()) continue;
                String[] parts = line.split(",");
                if (parts.length < 5) continue;

                String type = parts[0].trim();
                String code = parts[1].trim();
                String name = parts[2].trim();
                double price = Double.parseDouble(parts[3].trim());

                switch (type.toLowerCase()) {
                    case "aokhoac":
                        boolean haveHat = Boolean.parseBoolean(parts[4].trim());
                        them(new AoKhoac("AoKhoac", code, name, price, haveHat));
                        break;
                    case "giaydep":
                        int szGiay = Integer.parseInt(parts[4].trim());
                        them(new GiayDep("GiayDep", code, name, price, szGiay));
                        break;
                    // ... các sản phẩm khác
                }
            }
        } catch (IOException e) {
            System.out.println("Da xay ra loi khi doc du lieu tu file.");
        }
    }
```

```
public void ghiVaoFile(String fileName) {
        try (BufferedWriter writer = new BufferedWriter(
                new OutputStreamWriter(new FileOutputStream(fileName), StandardCharsets.UTF_8))) {
            for (SanPhamThoiTrang sp : danhSach) {
                writer.write(sp.getStringSaveFile());
                writer.newLine();
            }
            writer.flush();
        } catch (IOException e) {
            System.out.println("Da xay ra loi khi luu du lieu vao file.");
```

```
        }
    }
```

- **Kỹ thuật chính:**
- `sp.getStringSaveFile()` : Gọi đa hình hàm tạo chuỗi của từng đối tượng để ghi file.
- **Kỹ thuật phụ trợ:**
- `Try-with-resources` : Tự động giải phóng tài nguyên luồng đọc/ghi.
- `StandardCharsets.UTF_8` : Bắt buộc mã hóa UTF-8 bảo toàn tiếng Việt có dấu.
- Khối `try-catch (NumberFormatException)` khi đọc đơn giá: Nếu có dòng bị lỗi số thì chương trình bỏ qua dòng đó chứ không bị crash đột ngột.

3.3. Quan hệ Chứa đựng trong `ChiTietHoaDon` và `HoaDon`

```
public class ChiTietHoaDon {
    private SanPhamThoiTrang sanPham;
    private int soLuong;

    public void tangSoLuong(int amount) {
        if (amount > 0) this.soLuong += amount;
    }

    public double tinhThanhTienSauGiam(double phanTramGiamGia) {
        if (sanPham == null) return 0;
        double giaSauGiam1SP = sanPham.calculateAfterDiscount(phanTramGiamGia);
        return (giaSauGiam1SP + sanPham.VATcalculation()) * soLuong;
    }
}
```

- **Kỹ thuật chính (Aggregation):** Mỗi dòng hóa đơn giữ 1 con trỏ đến đối tượng sản phẩm và biến `soLuong` .
- **Kỹ thuật phụ trợ:** Hàm `tangSoLuong` : Khi khách hàng thêm món hàng đã có trong hóa đơn, chương trình cộng dồn số lượng chứ không thêm phần tử trùng vào danh sách.

3.4. Các thành phần Static trong hệ thống

```
public static final double TI_LE_VAT = 0.10;
private static int tongSoLuongSanPhamDaTao = 0;
private static int tongSoHoaDon = 0;
```

- **Kỹ thuật chính:**
- `TI_LE_VAT` : Hằng số tĩnh đại diện 10% thuế VAT.
- `tongSoLuongSanPhamDaTao` : Tăng lên khi `new SanPhamThoiTrang` .
- `tongSoHoaDon` : Biến static trong lớp `HoaDon` , tự động tăng khi hoàn tất thanh toán 1 hóa đơn mới, phục vụ việc thống kê doanh số.

4. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU (KÈM DÒNG CODE VÀ CÂU TRẢ LỜI ĐIỂM 10)

> **CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Lớp mảng các đối tượng là gì? Trong đồ án nhóm em cài đặt lớp mảng ở**

đâu và tại sao không dùng mảng tĩnh `SanPhamThoiTrang[]`?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:

1. Lớp mảng (Class Array / Collection Wrapper) là một lớp hướng đối tượng đóng gói một mảng hoặc danh sách bên trong, đồng thời cung cấp các phương thức nghiệp vụ tương ứng như Thêm, Sửa, Xóa, Tìm kiếm, Đọc/Ghi file. Trong đồ án, nhóm em cài đặt 2 lớp mảng chuyên biệt là `DanhSachSanPham` và `DanhSachKhachHang`.

2. Nhóm không dùng mảng tĩnh nguyên thủy `SanPhamThoiTrang[]` vì mảng tĩnh có kích thước cố định, khi thêm sản phẩm vượt quá số lượng sẽ bị lỗi tràn mảng hoặc phải viết hàm cấp phát lại bộ nhớ thủ công rất tốn kém. Việc đóng gói `ArrayList` trong lớp mảng vừa tận dụng được khả năng tự co giãn kích thước linh hoạt, vừa đảm bảo tính đóng gói của OOP."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào dòng `new InputStreamReader(..., StandardCharsets.UTF_8)` và hỏi: "Tại sao phải chỉ định `StandardCharsets.UTF_8`? Nếu bỏ đi có chạy được không?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:

1. Nếu bỏ `StandardCharsets.UTF_8` và chỉ dùng `new FileReader(file)`, Java sẽ lấy bảng mã mặc định của hệ điều hành máy tính đang chạy (trên Windows thường là Windows-1258 hoặc Windows-1252). Khi đó, các chuỗi tiếng Việt có dấu như 'Áo gió thể thao', 'Giày dép Biti's' sẽ bị lỗi font, biến thành ký tự lạ hoặc dấu hỏi chấm (`??`).

2. Việc chỉ định tường minh `StandardCharsets.UTF_8` ép buộc máy ảo Java đọc đúng định dạng Unicode 8-bit quốc tế, đảm bảo chương trình chạy đồng nhất và hiển thị tiếng Việt hoàn hảo trên mọi hệ điều hành (Windows, Linux, macOS)."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Cú pháp `try (BufferedReader reader = ...)` khác gì với khối `try-catch` bình thường?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, đây là cấu trúc **Try-with-resources** được giới thiệu từ Java 7:

1. Ở `try-catch` truyền thống, lập trình viên phải tự viết lệnh `reader.close()` trong khối `finally` và phải bọc thêm một `try-catch` nữa rất rườm rà. Nếu quên đóng, file sẽ bị khóa (file lock) và gây rò rỉ tài nguyên hệ thống.

2. Ở `Try-with-resources`, bất kỳ luồng nào khai báo trong ngoặc đơn của `try` sẽ được Java **tự động gọi hàm `close()` ngay khi khối lệnh kết thúc**, kể cả khi xảy ra Exception. Cách viết này vừa ngắn gọn, vừa an toàn tuyệt đối cho hệ thống."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Phân biệt thuộc tính Static và thuộc tính thông thường? Biến `tongSoHoaDon` hoạt động thế nào?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:

1. Thuộc tính thông thường thuộc về từng đối tượng cụ thể (Instance variable). Mỗi lần gọi `new`, một bản sao vùng nhớ mới được cấp phát riêng.

2. Thuộc tính Static thuộc về mức Lớp (Class-level). Toàn bộ chương trình chỉ có duy nhất một bản sao vùng nhớ dùng chung cho tất cả các đối tượng.

3. Biến `HoaDon.tongSoHoaDon` là static, nên dù trong phiên làm việc chúng em tạo 100 hóa đơn khác nhau, biến này vẫn được chia sẻ chung và tự động tăng dần để đếm chính xác tổng số đơn hàng đã phục vụ."

TÀI LIỆU CHI TIẾT DÀNH CHO TRƯỞNG NHÓM & BẢO VỆ VẤN ĐÁP (ĐIỂM 10/10)

> **Phân công phụ trách:** Thành viên 4 (Trưởng nhóm / Người thuyết trình & Demo chính)

> **Mục tiêu:** Nắm toàn bộ kiến trúc điều phối hệ thống (Facade Pattern), làm chủ vòng lặp giao diện Console Menu 10 chức năng (CRUD), quy trình bán hàng - in phiếu bảo hành tự động, kịch bản demo 7 bước mượt mà và bộ câu hỏi vấn đáp ăn trọn điểm 10 của Thầy Hiếu.

1. DANH MỤC CÁC TẬP TIN (FILES) QUẢN LÝ TRONG PHẦN 4

Phần 4 gồm tổng cộng **3 tập tin mã nguồn (.java)**:

1.1. Các tập tin mã nguồn chính (Core Files) - 2 Lớp

30. `QuanLyCuaHang.java` : Lớp điều phối trung tâm (Controller / Facade Pattern) kết nối giữa giao diện và các kho dữ liệu.
31. `Main.java` : Lớp giao diện dòng lệnh Console (View & Menu Dispatcher), tiếp nhận mọi tương tác của người dùng.

1.2. Tập tin kiểm thử tự động (Testing & Verification) - 1 Lớp

32. `TestCases.java` : Bộ kịch bản kiểm thử tự động gồm **73 testcase** kiểm tra độ bền vững, đa hình, kế thừa, ngoại lệ và tính đúng đắn của toàn bộ đồ án.

2. BẢNG PHÂN LOẠI KĨ THUẬT LẬP TRÌNH: CHÍNH VS PHỤ TRỢ

Phân loại	Tên kĩ thuật	Vị trí xuất hiện trong mã nguồn	Vai trò & Ý nghĩa kỹ thuật
Kỹ thuật chính	**Mô hình Điều phối Facade (Facade Pattern)**	Lớp `QuanLyCuaHang`	Che giấu sự phức tạp của các kho dữ liệu bên dưới; gom toàn bộ `DanhSachSanPham`, `DanhSachKhachHang`, `NhaCungCap` vào 1 đầu mối duy nhất.
Kỹ thuật chính	**Điều phối giao diện Menu (Menu Dispatcher)**	Vòng lặp `while (true)` & `switch-case` trong `Main.java`	Phân luồng điều khiển 10 chức năng hệ thống; đảm bảo ứng dụng chạy liên tục cho đến khi người dùng chọn thoát.
Kỹ thuật chính	**Quy trình Thanh toán & Xuất hóa đơn**	Case 8 trong `Main.java`, phương thức `displayBill()`, `xuatHoaDonRaFile()`	Tích hợp toàn diện: Chọn nhiều mặt hàng, cộng dồn số lượng, tra cứu giảm giá VIP/Nhân viên, tính VAT 10%, in phiếu bảo hành và lưu file text.
Kỹ thuật chính	**Kiểm thử tự động (Automated Unit Testing)**	`TestCases.java` (73 testcase)	Minh chứng tính đúng đắn khoa học của đồ án trước giảng viên: Không lỗi logic, không crash ngoại lệ, 100%

			testcase PASS.
Kỹ thuật phụ trợ	**Chống lỗi trôi lệnh Scanner**	`scanner.nextLine().trim()` kết hợp `Integer.parseInt`	Tránh triệt để lỗi bỏ qua dòng (Skip line) khét tiếng của `scanner.nextInt()` trong Java khi nhấn phím Enter.
Kỹ thuật phụ trợ	**Ép kiểu Downcasting có điều kiện**	`if (oldSp instanceof AoKhoac) { ((AoKhoac) oldSp).setHasHat(...); }`	Ép kiểu an toàn từ lớp cha về lớp con để sửa các thuộc tính đặc thù (mũ áo, size áo, chất liệu, size giày).
Kỹ thuật phụ trợ	**Cơ chế Tự động Lưu tức thì (Auto-save)**	Hàm `saveDataIntoFile()` trong `QuanLyCuaHang.java`	Tự động ghi đè dữ liệu mới nhất xuống file text sau mỗi thao tác Thêm, Sửa, Xóa; không bao giờ mất dữ liệu.
Kỹ thuật phụ trợ	**Tìm kiếm kép (Dual Search)**	Tìm theo mã chính xác & tìm theo từ khóa tên gần đúng	Tăng trải nghiệm người dùng, giúp tra cứu nhanh chóng khi không nhớ chính xác mã sản phẩm.

3. PHÂN TÍCH CHI TIẾT TỪNG ĐOẠN CODE: CHÍNH VS PHỤ TRỢ

3.1. Lớp điều phối trung tâm `QuanLyCuaHang.java`

```
public class QuanLyCuaHang {
    private DanhSachSanPham dsSanPham;
    private DanhSachKhachHang dsKhachHang;
    private ArrayList<NhaCungCap> dsNhaCungCap;

    public QuanLyCuaHang() {
        this.dsSanPham = new DanhSachSanPham();
        this.dsKhachHang = new DanhSachKhachHang();
        this.dsNhaCungCap = new ArrayList<>();
        khoiTaoNhaCungCapMacDinh();
    }

    public void addProduct(SanPhamThoiTrang product) {
        if (dsSanPham.them(product)) {
            saveDataIntoFile();
        }
    }

    public void saveDataIntoFile() {
        dsSanPham.ghiVaoFile("products.txt");
        dsKhachHang.ghiVaoFile("users.txt");
    }

    public void loadDataFromFile(String fileName) {
        dsSanPham.docTuFile(fileName);
        dsKhachHang.docTuFile("users.txt");
    }
}
```

- **Kỹ thuật chính (Facade Pattern):**
- Gom 3 kho dữ liệu: `dsSanPham` , `dsKhachHang` , `dsNhaCungCap` .
- Hàm `main` chỉ cần tạo duy nhất 1 đối tượng `QuanLyCuaHang store = new QuanLyCuaHang()` . Mọi thao tác thêm sản phẩm, thêm khách hàng, thanh toán đều gọi qua `store` .
- **Kỹ thuật phụ trợ (Auto-save):**
- Trong `addProduct` , `removeProduct` , `updateProduct` : Bất cứ khi nào thao tác mảng trả về `true` , hệ thống tự động gọi ngay `saveDataIntoFile()` .

3.2. Kỹ thuật tiếp nhận dữ liệu an toàn trong `Main.java`

```
while (true) {
        inMenu();
        System.out.print("Nhap lua chon cua ban: ");
        String input = scanner.nextLine().trim();
        int choice = -1;
        try {
            choice = Integer.parseInt(input);
        } catch (NumberFormatException e) {
            System.out.println("Vui long nhap so hop le!");
            continue;
        }
        // ... xử lý switch case
    }
```

- **Kỹ thuật chính:** Vòng lặp Menu chính điều khiển toàn bộ ứng dụng.
- **Kỹ thuật phụ trợ (Phòng chống lỗi nhập liệu):**
- Sử dụng `scanner.nextLine().trim()` thay vì `scanner.nextInt()` : Khi dùng `nextInt()` , phím Enter (`\n`) vẫn còn lưu lại trong bộ đệm (Buffer), làm cho lệnh `nextLine()` kế tiếp bị trôi qua mà không cho người dùng kịp nhập. Cách đọc chuỗi rồi ép kiểu bằng `Integer.parseInt` loại bỏ hoàn toàn lỗi này.
- Khối `try-catch (NumberFormatException)` : Nếu người dùng lỡ tay gõ chữ cái thay vì số, chương trình bắt lỗi nhẹ nhàng và nhắc nhở chứ không bị crash đột ngột.

3.3. Chức năng Sửa toàn diện kết hợp Downcasting (Menu 4)

```
if (editOpt.equals("4")) {
        if (oldSp instanceof AoKhoac) {
            System.out.println("Ao co non khong? (1: Co / 0: Khong): ");
            boolean hasHat = scanner.nextLine().trim().equals("1");
            ((AoKhoac) oldSp).setHasHat(hasHat);
        } else if (oldSp instanceof QuanJean) {
            System.out.println("Nhap kieu dang moi: ");
            String st = scanner.nextLine().trim().replace(",", " ");
            ((QuanJean) oldSp).setStyle(st);
        } else if (oldSp instanceof AoThun) {
            System.out.println("Nhap kich co moi (S/M/L/XL): ");
            String sz = scanner.nextLine().trim();
            ((AoThun) oldSp).setSize(sz);
        } else if (oldSp instanceof GiayDep) {
            System.out.println("Nhap size giay moi (35 - 45): ");
            int sg = Integer.parseInt(scanner.nextLine().trim());
            ((GiayDep) oldSp).setSizeGiay(sg);
        }
        store.saveDataIntoFile();
        System.out.println("Da cap nhat thuoc tinh rieng thanh cong!");
    }
```

- **Kỹ thuật chính (Downcasting & Instanceof):**
- Biến `oldSp` ban đầu có kiểu khai báo là lớp cha `SanPhamThoiTrang` (chỉ có các phương thức chung như `getTenSP` , `getGiaBan`). Lớp cha không hề có các phương thức `setHasHat` hay `setSizeGiay` .
- Nhờ toán tử `instanceof` kiểm tra đúng kiểu thực tế và cú pháp ép kiểu xuống `((AoKhoac) oldSp)` , chương trình có thể gọi được các phương thức riêng biệt của từng lớp con.

3.4. Quy trình Bán hàng, Chiết khấu & In Bảo hành (Menu 8)

```
HoaDon newbill = new HoaDon(newBillCode, null);
    // Nhập sản phẩm và số lượng...
    SanPhamThoiTrang product = store.getProductByCode(pCode);
    newbill.addProduct(product, sl);

    // Nhập số điện thoại khách hàng:
    Nguoi person = store.getVIPOrStaffByPhone(sdt);
    newbill.setPerson(person);

    // In hóa đơn và lưu file:
    newbill.displayBill();
    newbill.xuatHoaDonRaFile("hoadon.txt");
```

```
// Bên trong HoaDon.java: In bảo hành tự động
    public void printAllWarrantyCards() {
        for (ChiTietHoaDon ct : chiTietList) {
            SanPhamThoiTrang sp = ct.getSanPham();
            if (sp instanceof IBaoHanh) {
                ((IBaoHanh) sp).printWarrantyCard();
            }
        }
    }
```

- **Kỹ thuật chính:**
- `newbill.setPerson(person)` : Thiết lập đối tượng thanh toán.
- Trong `displayBill()` : Tự động gọi đa hình `((IGiamGiaKhachHang) person).applyDiscount(total)` nếu khách là VIP (giảm 20%), Thân thiết (giảm 10%), hoặc Nhân viên (giảm 50%).
- `printAllWarrantyCards()` : Tự động in phiếu bảo hành cho những món đồ cài đặt `IBaoHanh` .
- `xuatHoaDonRaFile("hoadon.txt")` : Ghi lại toàn bộ nội dung hóa đơn và phiếu bảo hành vào file nhật ký.

4. KỊCH BẢN DEMO ĐỒ ÁN TRƯỚC GIẢNG VIÊN (7 BƯỚC ĐẠT ĐIỂM 10)

33. **Bước 1 (Tổng quan kiến trúc - 30 giây):** Thưa Thầy, đồ án nhóm em xây dựng hệ thống Quản lý cửa hàng thời trang với **20 lớp đối tượng và 3 interface**, tổ chức theo mô hình phân tầng: Model Sản phẩm, Model Con người, Lớp mảng danh sách đóng gói, Lớp điều phối Facade và View Console.
34. **Bước 2 (Xem danh mục hàng hóa - Phím 5):** Bấm phím 5 hiển thị toàn bộ kho hàng dạng bảng Excel, có phân cột rõ ràng và định dạng tiền tệ chuẩn Việt Nam (`1.800.000 VND`).
35. **Bước 3 (Thêm sản phẩm tự sinh mã - Phím 1):** Chọn thêm một đôi Giày dép mới. Cho Thầy thấy hệ thống **tự động sinh mã `GD002`** mà người dùng không cần gõ mã bằng tay.
36. **Bước 4 (Tìm kiếm sản phẩm - Phím 3):** Demo 2 chế độ tìm kiếm: Tìm kiếm theo mã (`AK001`) và Tìm kiếm gần đúng theo tên (`Hunter` hoặc `Polo`).

37. **Bước 5 (Sửa thông tin sản phẩm - Phím 4):** Chọn sản phẩm vừa thêm và sửa thuộc tính riêng (ví dụ sửa size giày từ 40 lên 42). Mở file `products.txt` minh chứng dữ liệu được cập nhật ngay lập tức.
38. **Bước 6 (Bán hàng đa hình & In bảo hành - Phím 8):**

- Mua 2 Áo khoác (`AK001`) và 1 Áo thun (`AT001`).
- Nhập SĐT khách hàng VIP (`0912345678`).
- Bảng hóa đơn tự động tính VAT 10%, giảm giá 20% cho VIP và **tự động in kèm Phiếu bảo hành 12 tháng** cho Áo khoác.
- Mở file `hoadon.txt` cho Thầy xem hóa đơn được lưu trữ vĩnh viễn.

39. **Bước 7 (Minh chứng kiểm thử tự động - 1 phút):** Chạy lệnh `java TestCases` trên Terminal, hiển thị kết quả **73/73 testcase vượt qua tuyệt đối (0 thất bại)**.

5. BỘ 10 CÂU HỎI VẤN ĐÁP KINH ĐIỂN CỦA THẦY HIẾU (KÈM CÂU TRẢ LỜI ĐIỂM 10)

40. **Thầy hỏi: Đồ án có đủ 15 lớp theo yêu cầu không? Kể tên?**

Trả lời: Dạ thưa Thầy, đồ án của nhóm em có tổng cộng **20 lớp và 3 interface**. Gồm 7 lớp sản phẩm thời trang, 6 lớp phân cấp con người, 4 lớp quản lý và danh sách mảng, 2 lớp hóa đơn chi tiết, 1 lớp thực thi `Main` và 3 interface nghiệp vụ.

41. **Thầy hỏi: Lớp `QuanLyCuaHang` đóng vai trò gì trong kiến trúc hướng đối tượng?**

Trả lời: Dạ thưa Thầy, `QuanLyCuaHang` áp dụng mẫu thiết kế **Facade Pattern (Bộ điều phối trung tâm)**. Nó đóng gói các kho dữ liệu rời rạc bên dưới (`DanhSachSanPham` , `DanhSachKhachHang` , `NhaCungCap`) thành một giao diện đơn giản duy nhất để lớp `Main` gọi tới, giúp giảm sự phụ thuộc (Loose Coupling) giữa giao diện người dùng và cơ sở dữ liệu.

42. **Thầy hỏi: Đa hình được thể hiện ở những vị trí nào quan trọng nhất?**

Trả lời: Thể hiện ở 3 vị trí cốt lõi: (1) Đa hình qua lớp trừu tượng `SanPhamThoiTrang` với các hàm `getThongTinRieng()` , `getStringSaveFile()` , (2) Đa hình bảo hành qua `IBaoHanh.printWarrantyCard()` , (3) Đa hình chiết khấu khách hàng/nhân viên qua `IGiamGiaKhachHang.applyDiscount()` .

43. **Thầy hỏi: Lớp mảng danh sách đối tượng trong đồ án là gì?**

Trả lời: Nhóm xây dựng 2 lớp mảng chuyên biệt là `DanhSachSanPham` và `DanhSachKhachHang` . Chúng đóng gói `ArrayList` bên trong và cung cấp trọn vẹn các phương thức nghiệp vụ CRUD (Xem, Thêm, Sửa, Xóa, Tìm kiếm) và Đọc/Ghi file text.

44. **Thầy hỏi: Tại sao trong `Main.java` nhóm không dùng `scanner.nextInt()` để đọc lựa chọn menu?**

Trả lời: Vì `scanner.nextInt()` chỉ đọc số mà để lại ký tự Enter (`\n`) trong bộ đệm bàn phím, dẫn đến các lệnh `scanner.nextLine()` tiếp theo bị trôi qua mà không nhận được dữ liệu của người dùng. Nhóm sử dụng `scanner.nextLine().trim()` kết hợp `Integer.parseInt` để triệt tiêu hoàn toàn lỗi trôi lệnh.

45. **Thầy hỏi: Kỹ thuật ép kiểu Downcasting được áp dụng ở đâu?**

Trả lời: Được áp dụng khi sửa thuộc tính riêng của sản phẩm: `((AoKhoac) oldSp).setHasHat(hasHat)` và khi in phiếu bảo hành: `((IBaoHanh) sp).printWarrantyCard()` , sau khi đã dùng `instanceof` kiểm tra an toàn.

46. **Thầy hỏi: Nhóm xử lý lỗi font tiếng Việt khi đọc/ghi file như thế nào?**

Trả lời: Nhóm sử dụng `InputStreamReader` và `OutputStreamWriter` kết hợp chỉ định tường minh bảng mã quốc tế `StandardCharsets.UTF_8` cùng luồng đệm `BufferedReader` / `BufferedWriter` , đảm bảo hiển thị chuẩn 100% tiếng Việt trên mọi hệ điều hành.

47. **Thầy hỏi: Biến static `HoaDon.tongSoHoaDon` có ý nghĩa gì?**

Trả lời: Biến static là biến thuộc về mức Lớp, chỉ lưu duy nhất 1 vùng nhớ chia sẻ cho mọi thực thể hóa đơn, tự động tăng lên sau mỗi lần xuất đơn để thống kê chính xác tổng số hóa đơn đã phục vụ trong phiên làm việc.

48. **Thầy hỏi: Khi khách hàng mua trùng một mã sản phẩm nhiều lần trong 1 đơn hàng thì hệ thống xử lý sao?**

Trả lời: Lớp `HoaDon` duyệt giỏ hàng bằng hàm tìm kiếm theo mã; nếu đã có, hệ thống gọi phương thức `ct.tangSoLuong(sl)` để cộng dồn số lượng chứ không tạo thêm dòng mới, đảm bảo tính thẩm mỹ và nghiệp vụ của hóa đơn.

49. **Thầy hỏi: Nếu ứng dụng bị mất điện tắt đột ngột thì dữ liệu có bị mất không?**

Trả lời: Dạ không, vì hệ thống áp dụng cơ chế tự động đồng bộ tức thì (Auto-save). Bất kỳ thao tác thêm, xóa, sửa sản phẩm hay người dùng đều được ghi đè ngay lập tức xuống `products.txt` và `users.txt` .